

Experimental search for gap sequences for Shellsort

Introduction of the problem and State of the Art

Thesis / Speaker: inż. Adam Cherek
Supervisor: dr hab. inż. Paweł Czarnul



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

What is Shellsort?

- In 1959 Donald L. Shell in the article “A High-Speed Sorting Procedure” proposed his own sorting algorithm – later known as Shellsort
- Shellsort was created as a response to the hardware limitations of the time
- The idea was to mix the speed benefits of merging by pair, and the space benefits of sifting (shifting and insertion)
- To this day Shellsort finds its applications in:
 - Embedded systems
 - Introspective sorts as a sub-algorithm preventing from going too deep in recursion
 - Bzip2 compressor
 - Homomorphic encryption

Operation of Shellsort

- Shellsort can be described as a variation of Insertion Sort.
- It's *in-place*, *adaptive*, *comparison-based* algorithm, but due to its implementation – Shellsort is *not stable*.
- Unique part of Shellsort is a *gap sequence*

ALGORITHM Shellsort(array a, integer n)

// Define gap sequence in descending order

LIST gaps = [7, 4, 1]

// Iterate through each gap value

FOR EACH gap **IN** gaps **DO**

// Perform gapped insertion sort

FOR i = gap **TO** n-1 **DO**

// Save current element

temp = a[i]

// Shift elements that are larger than temp

j = i

WHILE j >= gap **AND** a[j - gap] > temp **DO**

a[j] = a[j - gap]

j = j - gap

END WHILE

// Insert temp at correct position

a[j] = temp

END FOR

END FOR

END ALGORITHM

Gap Sequences

Gap sequences are series of positive integers (known as gaps or increments) that dictate step length of the Shellsort algorithm. Each valid sequence for shell sort must contain 1 – to ensure that the output will indeed be sorted, as sorting iteration with a gap equal to 1 is equivalent to performing a standard Insertion Sort on the data – and the sequence should be monotonic.

Valid sequence:

{1, 3, 5, 9, 17, ...}

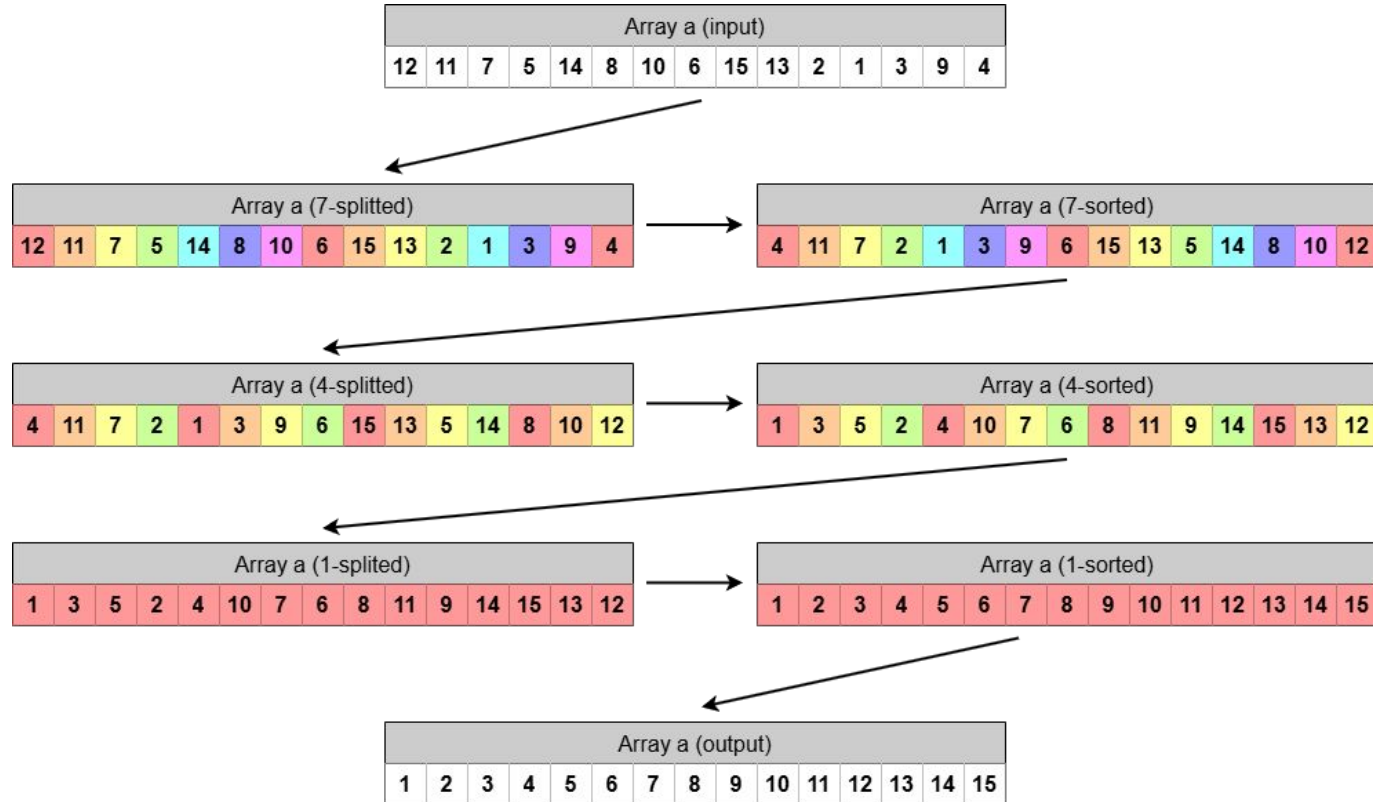
1. Starts with 1 – ensures that the output will be sorted
2. Monotonic sequence – ensures potential for being optimal

Invalid sequence:

{3, 9, 5, 17, ...}

1. Doesn't contain 1 – output might not be fully sorted
2. Non-monotonic sequence – a lot of meaningless comparisons due to H-sorting theorem

Flow of splitting and sorting in Shellsort



Finding the optimal Gap Sequence



D. Shell (1959) – first gap sequence

$$h_k = \lfloor \frac{N}{2^k} \rfloor \Rightarrow \{ \lfloor \frac{N}{2} \rfloor, \lfloor \frac{N}{4} \rfloor, \dots, 2, 1 \},$$

- First gap sequence emerged from the initially developed version of the algorithm
- Looks valid on paper, but hides a critical flaw:
 - Edge cases slower than standard Insertion Sort
 - A lot of directly redundant comparisons
- Faster on average but with worst case complexity of $O(N^2)$

Early improvements (1960-65)

Frank & Lazarus (1960)

- Noticed a critical flaw of Shell's sequence
- Proposed improvement by forcing all gaps to be odd numbers

$$h_k = 2 \lfloor \frac{N}{2^{k+1}} \rfloor + 1$$

Hibbard (1963)

- Favored Shellsort for small problems
- Believed that the right sequence could reach order of $N \log_2 N$ comparisons

$$h_k = 2^k - 1$$

Pepernov & Stasevich (1965)

- Proved worst-case complexity of $O(N^{3/2})$
- Linked Shellsort gap sequences with Frobenius problem

$$h_k = 2^k + 1, \text{ prefixed with } 1$$

V. Pratt (1972) – asymptotically best worst-case complexity

$$h_k = \left\{ \begin{array}{l} \text{successive numbers of form } 2^p 3^q, \\ \text{where } p, q \in \mathbb{Z}^+ \end{array} \right\} \Rightarrow \{1, 2, 3, 4, 6, 8, 9, \dots\},$$

- Up to this day asymptotically best worst-case complexity of $O(N \log^2 N)$
- Emphasized benefits of coprimeness between increments
- Few mathematical contributions like: The Sorting Template, “Non-Messing-Up” Theorem, and Complexity Metrics
- Categorization of gap sequences into First-Order and Higher-Order Progressions
- First mention of Shellsort parallelization

D. Knuth (1973) – back to reality

$$h_k = \left\{ \begin{array}{l} h_1 = 1 \\ h_k = 3h_{k-1} + 1 \end{array} \right\} = \frac{3^k - 1}{2} \Rightarrow \{1, 4, 13, 40, 121, \dots\},$$

- Acknowledged Pratt's sequence but noticed that it is hardly practical for real-world problems
- Proved that even a sequence of two gaps can result in average running time proportional to $N^{5/3}$
- Suggested simple solutions for practical purposes

R. Sedgwick (1983-86) – better on average bounds

$$h_k = 4^k + 3(2^{k-1}) + 1, \text{ prefixed with } 1 \Rightarrow \{1, 8, 23, 77, 281, \dots\},$$

$$h_k = 2(4^{k+2}) - 9(2^{k+2}) + 9, \text{ prefixed with } 1 \Rightarrow \{1, 5, 65, 377, 1769, \dots\},$$

- Improved worst-case complexity to $O(N^{4/3})$ while using only up to $\log N$ increments
- Working with J. Incerpi proved that it is possible to achieve $O(N^{1+4/\sqrt{2\lg N}})$ – in theory it means that some sequences can reach $O(N^{1+\epsilon})$
- Presented that the coprimality is not necessary, but used a generalized Frobenius function of two arguments

Beginning of an empirical era (1991-92)

Gonnet & Baeza-Yates
(1991)

- Worked on real execution times instead of theoretical factors
- Accounted for hidden costs like memory allocation and indexing
- First gap sequence derived by extensive simulations

$$h_k = \left\{ \begin{array}{l} h_0 = N, \alpha = 0.45454 \\ h_k = \max(\lfloor \alpha(h_{k-1}) \rfloor, 1) \end{array} \right\}$$

Tokuda
(1992)

- Established empirical trials as a search method
- His sequence became a benchmark for future gap sequences

$$h_k = \lceil \frac{1}{5} (9(\frac{9}{4})^{k-1} - 4) \rceil = \lceil \frac{(9/4)^k - 1}{(9/4) - 1} \rceil$$

M. Ciura (2001) – fully experimentally derived sequence

$\{1, 4, 10, 23, 57, 132, 301, 701\},$

- Derived sequence purely by experiments and sequential analysis, no general term h_k
- Defined traits of a good sequence:
 - Ratio – the best sequences generally have a growth ratio of subsequent increments between 2 and 3.
 - Modulo – modulo of subsequent increments equaling to 0 is considered a hindrance.
 - Avoiding simple recursions – sequences defined by simple linear recurrences (like Hibbard's or Knuth's) were found to be relatively bad because they lead to higher counts of redundant comparisons.
 - Interaction and redundancy – a "good" sequence minimizes comparisons between elements that have already been directly or indirectly compared in previous passes
- Proposed diophantine equation as a method to forecast sequence's performance
- Currently the fastest sequence for small input data

Y. Lee (2021) – improvement of Tokuda's sequence

$$h_k = \left\{ \begin{array}{l} h_k = \lceil \frac{\gamma^k - 1}{\gamma - 1} \rceil \\ \gamma = 2.243609061420001 \end{array} \right\} \Rightarrow \{1, 4, 9, 20, 45, 102, 230, 516, \dots\}.$$

- Improved Tokuda's sequence by finding more optimal coefficient γ
- Found optimal gap sequences for input size $N < 16$
- Presented two technical innovations to help with the empirical research:
 - Parameterization by index – a systematic way to generate all 2^{N-2} valid gap sequences
 - Reduced permutational space – it is only necessary to test a proper “Bad (s,1)-sorted” subset instead of whole $N!$ permutations
- Calculated that the thorough exploration of all possible solutions for real-life problems size would be too exhaustive for current technical capabilities, even with presented optimizations

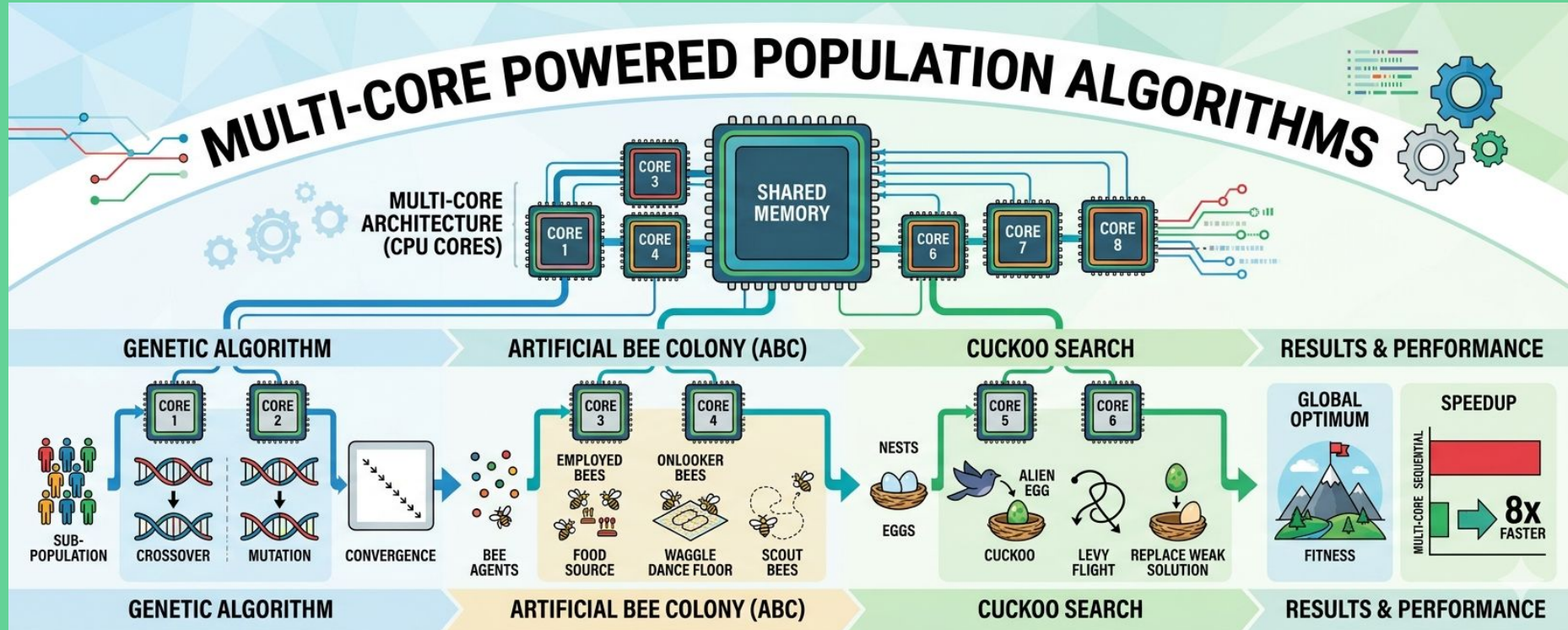
Skean & Ehrenborg & Jaromczyk (2023) – state of the art

$$k_A(i) = \lfloor (a^{\lfloor i/b \rfloor} \cdot c^{\lfloor i/d \rfloor})^f + e \rfloor, \quad k_B(i) = \lfloor (a \cdot b^{\lfloor i/c \rfloor}) + d \rfloor,$$

- Derived two template functions (differing in number of parameters), based on their observations of best performing gap sequences
- Utilized a grid-search and sequential analysis for the purpose of finding optimal parameters filling their template functions (different depending on assessment criterion)
- Almost as good sequence as Ciura's for small input, but with advantage of general term h_k derivation
- Most likely currently the best function-based gap sequences (unfortunately not officially compared with Lee's)

The Solution(?):

Population-based algorithms powered by multi-core architecture



Population-based algorithms

- Searching/Optimizing algorithms like:
 - Genetic Algorithm (GA)
 - Artificial Bee Colony (ABC)
 - Cuckoo Search (CS)
- Adjustment and optimization for the gap sequences problem
- Huge statistical sample and analysis of the results

Multi-core architecture

- High-performance computing in C++ with CUDA and OpenMP libraries
 - Modern architecture utilizing several CPUs and GPUs
 - Optimization of simulation processes to match the hardware capabilities
-

Population based approach

- Some swarm intelligence algorithms are theoretically well designed for the problem of exploration and exploitation of Shellsort gap sequences solution space
- Genetic Algorithm provides good base for adjustments for chosen problem
- Optimization and satisfaction with suboptimal solutions still seem necessary, given the size of problem and current technological limits

Decided solution:

Genetic Algorithm with quick extinction of previous generations, reinforced by optimization methods from ABC and Cuckoo Search.

Acceleration with Multi-core architecture and HPC

- Even with optimizations and metaheuristics algorithms, the problem still remains too large for a simple “home solution”
- As the problem requires quantity not quality, the solution will be to perform as many runs of Shellsort with different gap sequences at the same time as possible

<i>N</i>	Full space	Reduced space
1 000	2.67877e+300	3.98303e+40
10 000	4,98766e+3009	4.91871e+85

Decided solution:

Orchestration of populations creation, read/write operations, and GPUs utilization by OpenMP, and thousands of Shellsort procedures performed with CUDA.

Bibliography

- ❖ Ciura, M. (2001). Best increments for the average case of Shellsort. In R. Freivalds (Ed.), *Fundamentals of Computation Theory* (pp. 106–117). Berlin, Germany: Springer.
- ❖ Frank, R. M., & Lazarus, R. B. (1960). A high-speed sorting procedure. *Communications of the ACM*, 3(1), 20–22.
- ❖ Gonnet, G. H., & Baeza-Yates, R. (1991). *Handbook of algorithms and data structures: In Pascal and C* (2nd ed.). Reading, MA: Addison-Wesley.
- ❖ Hibbard, T. N. (1963). An empirical study of minimal storage sorting. *Communications of the ACM*, 6(5), 206–213.
- ❖ Incerpi, J., & Sedgewick, R. (1985). Improved upper bounds on Shellsort. *Journal of Computer and System Sciences*, 31(2), 210–224.
- ❖ Lee, Y. W. (2021). Empirically improved Tokuda gap sequence in Shellsort. arXiv preprint arXiv:2112.12450.
- ❖ Lee, Y. W. (2021). Optimal gap sequences in Shellsort for $n \leq 16$ elements. arXiv preprint arXiv:2104.05389.
- ❖ Papernov, A. A., & Stasevich, G. V. (1965). A method of information sorting in computer memories. *Problems of Information Transmission*, 1(3), 63–75 [Translation from *Problemy Peredachi Informatsii*, 1(3), 81–98].
- ❖ Pratt, V. R. (1972). *Shellsort and sorting networks* (Technical Report STAN-CS-72-260). Stanford, CA: Stanford University, Department of Computer Science.
- ❖ Sedgewick, R. (1986). A new upper bound for Shellsort. *Journal of Algorithms*, 7(2), 159–173.
- ❖ Sedgewick, R. (1990). *Algorithms in C*. Reading, MA: Addison-Wesley.
- ❖ Shell, D. L. (1959). A high-speed sorting procedure. *Communications of the ACM*, 2(7), 30–32.
- ❖ Skean, O., Ehrenborg, R., & Jaromczyk, J. W. (2021). *Optimization perspectives on Shellsort*. Lexington, KY: University of Kentucky.
- ❖ Tokuda, N. (1992). An improved Shellsort. In *Proceedings of the IFIP 12th World Computer Congress on Algorithms, Software, Architecture-Information Processing* (Vol. 1, pp. 449–457).

The End

Thank you for your attention.



Presentation by inż. Adam Cherek, s207250